



Automating Bitvector and Finite Field Equivalence Proofs in Lean

Elizaveta Pertseva¹, Valentin Robert², Clark Barrett¹,
and James Parker²

¹ Stanford University, Stanford, USA

`pertseva@stanford.edu`

² Galois, Portland, USA



Abstract. Efforts to verify Zero-Knowledge Proof circuit encodings have highlighted the challenge of proving the correctness of quantifier-free statements that make use of both bitvector and finite field operations. Existing verification workflows are either manual or rely on SMT solvers, which scale poorly on some classes of problems for reasons that include difficulties with conversion operators and challenges reasoning about inequalities. To address these limitations, we present a novel Lean tactic `BitModEq` that leverages range lemmas and case analysis to produce verified translations from finite fields to bitvectors. Our approach, combined with bit-blasting, outperforms state-of-the-art SMT solvers, solving 19% more ZKP arithmetization benchmarks.

1 Introduction

Recent advances in Zero-Knowledge Proofs (ZKPs) have led to the utilization of ZKPs in critical applications such as cryptocurrencies [17], post-quantum signature schemes [7], voting systems [23], proofs of vulnerabilities [10, 11], and image authentication [12]. ZKPs are cryptographic algorithms that enable one party (the prover) to convince another party (the verifier) that some secret data satisfies a property ϕ without revealing any information about the data.

ZKP statements use the language of finite fields, where operations such as addition and multiplication are performed modulo a prime. However, real-world systems operate over concrete machine integers, using arithmetic modulo some power of two, also known as the theory of *bitvectors*. As a result, ZKP statements often require arithmetizations which encode bitvector operations as finite field operations. The correctness of these arithmetizations is crucial for the soundness of ZKP systems, but constructing them can be error-prone.

Prior work on verifying ZKP arithmetizations has followed two main approaches: manual and automatic. The manual approach typically works by embedding entire ZKP systems in interactive theorem provers (ITPs), as seen in formalizations of SP1 Hypercube and Cairo [3, 39]. While these efforts provide strong correctness guarantees, they require substantial human effort. Further, because ZKP optimizations evolve rapidly, arithmetizations frequently change, requiring significant portions of the verification effort to be revisited.

The automatic approach reduces verification to solving Satisfiability Modulo Theories (SMT) queries [30, 38]. While this approach enables greater automation, it poses two fundamental challenges that limit what can be done automatically. First, arithmetizations simultaneously involve bitvectors and finite fields, requiring the use of theory conversion operators that create performance problems for SMT solvers. Second, SMT solvers contain a much larger trusted code base (TCB) than ITPs, which increases the probability of missed bugs.¹

To address these challenges, we present **BitModEq**, a Lean tactic that translates finite field operations to bitvectors and solves the resulting goals via bit-blasting [8]. **BitModEq** retains the guarantees of ITPs and outperforms existing automatic methods. Our approach is guided by two main insights.

First, the majority of ZKP arithmetizations do not inherently rely on field-specific behavior (e.g., inverses or field division). Instead, they simulate bit-level operations inside the field, producing large numbers of inequality constraints that overwhelm the finite field reasoning engines. To remedy this, we propose automatically translating field elements back to bitvectors during verification, enabling efficient reasoning over the original bit-level semantics.

Our second insight is that the effectiveness of the translation primarily relies on range analysis. With tighter bounds, we can more frequently eliminate modulo operations, creating bitvectors of smaller widths. While Lean already has built-in lemmas for reasoning about equation ranges, they do not consider variable dependencies, which are crucial in bitvector operations simulating arithmetic. To remedy this, we design a Lean-specific range analysis that mixes Lean lemmas with targeted case reasoning. Our main contributions are as follows:

1. A Lean tactic **BitModEq** that translates from finite fields to bitvectors.
2. A proof-of-concept bounded verification of two real-world ZKP encodings that increases the bounds achieved by previous techniques.
3. A Lean range analysis tactic for proving goals resulting from the translation.

The remainder of the paper is organized as follows. We first present a motivating example (Sect. 2) and relevant background (Sect. 3). We then describe our translation algorithm (Sect. 4.2), range analysis tactic (Sect. 4.3), and Lean implementation (Sect. 4.4). We conclude with an evaluation (Sect. 5), related work (Sect. 6), and a discussion of future work (Sect. 7).

2 Motivating Example

Consider the polynomial shown in (1) over a finite field of order P , where P is larger than 2^B . The polynomial is an encoding of the **bvor** (bitvector OR) operation used in the Jolt zkVM [2]. In this encoding, x_i represents the i -th bit of one operand, and y_i represents the i -th bit of the other. We show all possible values for the polynomial and its subexpressions for the case $B = 1$ in Table 1.

¹ Work on emitting proof certificates from SMT solvers is one approach for addressing this [5], but proof support for the theory of finite fields remains ongoing research.

Table 1. Evaluation table for the finite field polynomial encoding (1) of **bvor** (bitwise OR) in the Jolt zkVM for 1-bit inputs [2].

x_0	y_0	$x_0 + y_0$	$x_0 \cdot y_0$	$2^0 \cdot (x_0 + y_0 - x_0 \cdot y_0)$
0	0	0	0	0
1	0	1	0	1
0	1	1	0	1
1	1	2	1	1

$$\sum_{i=0}^{B-1} 2^i (x_i + y_i - x_i \cdot y_i) \quad (1)$$

To verify that (1) correctly encodes the **bvor** operation, we must encode the relationship between the bitvector operation and the finite field polynomial (1). Inspired by prior work [30], we rely on a predicate **ff2bv**(bv, e) (read as bv is the bitvector encoding of the finite field element e) that encodes an equivalence between a bitvector bv of bit width N and a finite field term e . While the exact definition of **ff2bv** varies across systems [2, 27, 39], it most often includes a *range check*. In Jolt, **ff2bv**(bv, e) is defined if and only if the value of e is smaller than the target bitvector size, 2^N ; otherwise, the operation is undefined.² Let bv_1, bv_2 be bitvectors of bit width B , and let x_i, y_i be finite field elements from field P , for $i \in \{0, \dots, B-1\}$. Our full verification query becomes:

$$\frac{\bigwedge_{i \in \{0 \dots B-1\}} \mathbf{ff2bv}(bv_1[i], x_i) \quad \bigwedge_{i \in \{0 \dots B-1\}} \mathbf{ff2bv}(bv_2[i], y_i)}{\mathbf{ff2bv}(\mathbf{bvor} \ bv_1 \ bv_2, \sum_{i=0}^{B-1} 2^i (x_i + y_i - x_i \cdot y_i))} \quad (2)$$

One of the main focuses of this paper is the question of how to efficiently embed the **ff2bv** operator. Previous SMT-based approaches are bottlenecked by the **ff2bv** embedding for two main reasons. First, since SMT solvers do not support direct conversion from finite fields to bitvectors, the encoding must use bit splitting, which adds a large number of variables. Second, since current SMT theories of finite fields do not support inequalities directly [15], the SMT encoding must use a finite field polynomial $f(e) = \prod_{i=0}^{2^N-1} (e - i)$ for range checks,³ which slows down the solver. The full SMT encoding of **ff2bv** is as follows:

$$\begin{aligned} \mathbf{ff2bv}_{\text{SMT}}(bv, e) \iff & \prod_{i=0}^{2^N-1} (e - i) = 0 \wedge \exists e_0 \dots e_{N-1}. e = \sum_{i=0}^{N-1} 2^i e_i \\ & \wedge \bigwedge_{i=0}^{N-1} e_i(e_i - 1) = 0 \wedge \bigwedge_{i=0}^{N-1} bv[i] = \mathbf{ite}(e_i = 0, 0_{[1]}, 1_{[1]}), \end{aligned} \quad (3)$$

² Other systems may apply a modulo operation or return zero when e is too large.

³ $f(e)$ is zero on and only on $\{0, \dots, 2^N - 1\}$ and thus fixes the possible values of e to the interval $\{0, \dots, 2^N - 1\}$ [30].

where $c_{[B]}$ denotes a bitvector constant with value c and bit width B , and 2^N is smaller than the order of the finite field, as is the case for Jolt. A weaker SMT encoding can omit the range check polynomial, assuming instead that no overflow occurs during the conversion. If this assumption is correct, bit splitting already constrains the values of e appropriately. We refer to this as the *weak encoding*, as it relies on an unproved assumption. However, even the weak version of **ff2bv** remains challenging for SMT solvers, as bit splitting is still present.

In contrast, ITPs have richer semantics than SMT solvers, enabling direct conversion between different types, including from finite fields to natural numbers and from natural numbers to bitvectors. Inspired by manual proofs [3, 39], we rely on a **toNat**(e) function (present in most ITPs [25]) that maps a finite field term to its underlying natural number value. Then, the $f(e)$ polynomial becomes a simple range check over the natural numbers: $\text{toNat}(e) \leq 2^N - 1$. Similarly, instead of bit splitting, we can use **toBV**(N, t), a function that converts a natural number term t to a bitvector of size N . Our **ff2bv** encoding is thus:

$$\text{ff2bv}_{\text{ITP}}(bv, e) \iff \text{toNat}(e) < 2^N \wedge bv = \text{toBV}(N, \text{toNat}(e)). \quad (4)$$

Decoding **ff2bv** in (2) yields:

$$\frac{\begin{array}{l} \bigwedge_{i \in \{0, \dots, B-1\}} \text{bv1}[i] = \text{toBV}(1, \text{toNat}(x_i)) \wedge \text{toNat}(x_i) \leq 1 \\ \bigwedge_{i \in \{0, \dots, B-1\}} \text{bv2}[i] = \text{toBV}(1, \text{toNat}(y_i)) \wedge \text{toNat}(y_i) \leq 1 \end{array}}{(\text{bvor bv1 bv2}) = \text{toBV}\left(B, \text{toNat}\left(\sum_{i=0}^{B-1} 2^i(x_i + y_i - x_i \cdot y_i)\right)\right) \wedge \text{toNat}\left(\sum_{i=0}^{B-1} 2^i(x_i + y_i - x_i \cdot y_i)\right) \leq 2^B - 1} \quad (5)$$

While more compact, the ITP encoding raises the following question: to discharge the two goals, how can one relate the conversions appearing in the goal to those applied to variables in the premises?

Our solution is an algorithm that soundly distributes the operators **toNat** and **toBV** over finite field and natural number operations using range analysis. On the Jolt **OR** example, our approach, combined with bit-blasting, is able to automatically verify inputs with widths of up to 32 bits within five minutes and 8GB of memory. Under the same limits, the **cvc5** SMT solver only verifies inputs with widths up to 9 bits under the weak encoding and up to 3 under the full encoding.

3 Background

In this section, we introduce the notation and logical frameworks used throughout the paper. We first describe the different arithmetic domains we will reason about [13, 22]; we then discuss the logical setting we use [14]; and we conclude with an overview of Lean [25] and some existing Lean automation techniques [6, 8, 24]. More details can be found in the cited work.

Table 2. Relevant symbols from the theories referenced in this paper. For Σ_{BV} , symbols are parameterized by $N \in \mathbb{N}_{>0}$, which represents the bit width.

Signature	Sort	Set of Symbols
$\Sigma_{\mathbb{F}_P}$	$\mathbb{F}_P$	$\{\cdot_{\mathbb{F}_P}, +_{\mathbb{F}_P}, -_{\mathbb{F}_P}\}$
$\Sigma_{\mathbb{N}}$	$\mathbb{N}$	$\{\cdot_{\mathbb{N}}, +_{\mathbb{N}}, -_{\mathbb{N}}, \text{mod}_{\mathbb{N}}, \leq_{\mathbb{N}}, \geq_{\mathbb{N}}\}$
Σ_{BV}	BV_N	$\{\cdot_{[N]}, +_{[N]}, -_{[N]}, \text{mod}_{[N]}, \leq_{[N]}, \geq_{[N]}\}$
$\Sigma_{\mathbb{F}_P} \cup \Sigma_{\mathbb{N}}$	$\mathbb{F}_P, \mathbb{N}$	$\{\text{toNat}\}$
$\Sigma_{\mathbb{N}} \cup \Sigma_{\text{BV}}$	$\mathbb{N}, \text{BV}_N$	$\{\text{toBV}_{[N]}, \text{bvToNat}_{[N]}\}$

3.1 Arithmetic Domains

We reason about expressions over three arithmetic domains: natural numbers, finite fields, and bitvectors. We write $\mathbb{N}$ for the natural numbers (including 0) and $\mathbb{N}_{>0}$ for strictly positive naturals. In $\mathbb{N}$, subtraction is truncated: for $A, B \in \mathbb{N}$, the term $A - B$ is defined as usual unless $B > A$, in which case it is 0. We use $[i, j]$ to represent the set $\{n \in \mathbb{N} \mid i \leq n \leq j\}$ and $[n]$ to abbreviate $[1, n]$.

We use $\mathbb{F}_P$, with $P \in \mathbb{N}_{>0}$ prime, to denote a finite field of order P (we consider only prime-ordered fields). The field $\mathbb{F}_P$ is isomorphic to the natural numbers modulo P . We use the function $\text{toNat} : \mathbb{F}_P \rightarrow \mathbb{N}$ to return the representative in $\mathbb{N}$ of an element of $\mathbb{F}_P$.

We use BV_N with $N \in \mathbb{N}_{>0}$ to denote a bitvector of width N . Bitvectors can be viewed as finite sequences of bits, and we write $b[i]$ to denote the i -th bit of a bitvector b . Bitvectors have a canonical natural number representation in the range $[0, 2^N - 1]$. We use the function $\text{bvToNat} : \text{BV}_N \rightarrow \mathbb{N}$ to map a bitvector to its natural number representation and the function $\text{toBV} : \mathbb{N} \times \mathbb{N} \rightarrow \text{BV}_N$ to map a natural number to its N -bit encoding.

3.2 Logical Setting and Notation

Logic. We work in the logical setting of many-sorted quantifier-free first-order logic with equality, using standard terminology throughout. We use Σ to denote a many-sorted signature with a finite set of sorts, including Bool . We also assume a family of equality symbols $=_{\sigma} : \sigma \times \sigma \rightarrow \text{Bool}$ for each $\sigma \in \Sigma$. We assume the usual definitions of well-sorted terms and formulas (terms of sort Bool). We use v, w to denote variables, t to denote arbitrary terms, and γ to denote an arbitrary formula. A theory consists of a signature together with a fixed interpretation of some subset of its symbols.

We consider three base theories, each with a corresponding signature. The most relevant symbols are shown in Table 2. The finite field signature $\Sigma_{\mathbb{F}_P}$ includes the finite field sort, which we also denote $\mathbb{F}_P$, and the standard finite field operators [15]. The natural number signature $\Sigma_{\mathbb{N}}$ includes the natural number sort, which we also denote $\mathbb{N}$, and the usual arithmetic operators. The bitvector signature Σ_{BV} includes, for each $N \in \mathbb{N}_{>0}$, the bitvector sort for bitvectors of

size N , which we also denote $\mathbf{BV}_N$ and the SMT-LIB bitvector operators [34]. For all signatures, when the intended sorts of operators are clear from context, we write operators without explicit sort annotations.

We allow theory combination in the standard way. When working in the combined theory, we additionally assume the signature is extended with three *conversion operators* matching the three conversion functions (`toNat`, `toBV`, and `bvToNat`) mentioned above. For convenience, we refer to both the functions themselves and the operators with the same notation.

Notation. For a formula γ and terms s and t , we write $s \preceq \gamma$ to denote that s occurs as a subterm of γ . We write $\gamma[s/t]$ for the formula obtained by replacing all occurrences of s in γ with t . We lift this notation pointwise to sets of formulas and tuples of sets of formulas. For a set S of formulas, $S[s/t]$ denotes the set obtained by replacing all occurrences of s in S with t . Similarly, for a tuple $T = (S_1, S_2)$ of sets of formulas, we define $T[s/t] = (S_1[s/t], S_2[s/t])$. We also lift set membership to tuples of sets, using $\gamma \in T$ to denote $\gamma \in S_1 \cup S_2$. We write $S \setminus \gamma$ to abbreviate set $S \setminus \{\gamma\}$ and S, γ to abbreviate $S \cup \{\gamma\}$. Finally, we write $S := S'$ to denote assignment of S to S' and $S \models \gamma$ to indicate that γ is logically implied by S .

3.3 Lean and SMT

Lean. Lean [25] is an interactive theorem prover (ITP) that allows users to *mechanically* verify mathematical theorems, including those about properties of software. Lean provides a tactic language that enables users to interactively construct proofs by transforming proof states, which consist of a goal together with a local hypothesis context. Lean's type checker automatically verifies proofs for users and throws errors during compilation if a proof is not valid. This provides high confidence that the proof is correct, as it shrinks the trusted computing base (TCB) to Lean's small trusted kernel. Lean is based on higher-order logic and is therefore more expressive than the first-order setting described above. Our tactic operates on Lean goals but reasons in the restricted first-order setting, which is typically sufficient for ZKP arithmetizations. We thus use the notation of many-sorted first-order logic in this paper, with the understanding that we are working with a corresponding logical fragment within Lean.

SMT. Satisfiability Modulo Theories (SMT) solvers allow users to *automatically* verify theorems expressed in the language of many-sorted first-order logic with theories. Some commonly supported theories include uninterpreted functions, integer and real arithmetic, and arrays and bitvectors [6]. SMT solvers are more automated but less expressive than ITPs. To solve queries over multiple theories, SMT solvers employ dedicated theory solvers and theory combination mechanisms such as the Nelson-Oppen framework [26]. Since each theory solver operates independently, conversion operators are typically challenging to support and expensive for the solvers to reason about.

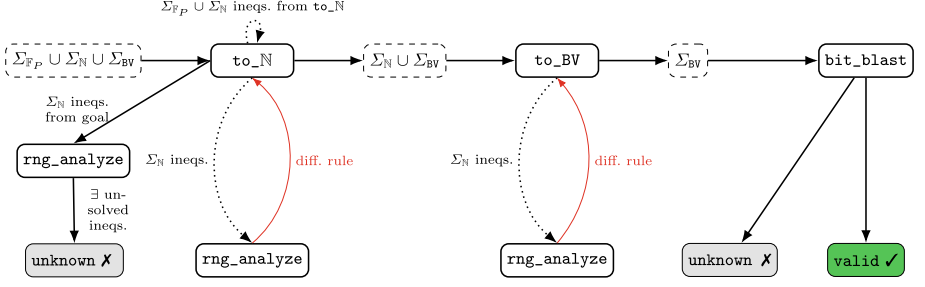


Fig. 1. BitModEq workflow. The tactic accepts terms over $\Sigma_{\mathbb{F}_P}$ and Σ_{BV} , and terms originating from **toNat** and **bvToNat** operators. It first converts $\mathbb{F}_P$ terms to $\mathbb{N}$ terms and then converts $\mathbb{N}$ terms to BV_N terms. Inequalities are discharged by **rng_analyze**. If **rng_analyze** cannot prove an inequality from the original goal, the tactic returns **unknown**; if it cannot prove a premise from a translation rule, a different rule is applied. Once only Σ_{BV} goals and hypotheses remain, the goals are discharged via bit-blasting. **valid** is returned if and only if bit-blasting solves the problem and no inequalities remain.

There are two main approaches for integrating SMT-like reasoning with Lean. The first involves translating Lean proof obligations into SMT queries, sending them to an external SMT solver, and then reconstructing the proof in Lean to convince the kernel of the result [24]. The reconstruction approach only supports a subset of SMT theories, not including (at the time of writing) finite fields and bitvectors. The second approach is to implement SMT-like decision procedures directly in Lean, which keeps the reasoning inside the proof assistant but may lead to increased solving times due to the need to perform the proof search within Lean. In this paper, we follow the second approach and rely on **bv_decide**, a bit-blasting decision procedure implemented in Lean [8].

4 BitModEq

In this section, we describe the BitModEq algorithm. We begin with a high-level overview and then examine its core elements: conversion and range analysis.

4.1 Overview

Figure 1 illustrates the BitModEq workflow, which operates in four phases:

1. Convert all terms of sort $\mathbb{F}_P$ in the proof context to terms of sort $\mathbb{N}$.
2. Discharge any inequalities over $\mathbb{N}$ in the original goal using range analysis.
3. Convert any remaining terms of sort $\mathbb{N}$ to bitvector terms.
4. Discharge the final bitvector equality goal via the **bv_decide** tactic.

Since finite field elements do not admit a canonical bitvector representation, we must select an intermediary domain. Two natural candidates are the integers and the natural numbers. We choose the natural numbers for two reasons. Primarily, in the majority of the ZKP arithmetizations, finite field variables are used to simulate bits and therefore do not represent negative values. Second, converting integers to bitvectors requires reasoning about both lower and upper bounds, whereas conversion from natural numbers requires reasoning only about upper bounds, eliminating a significant portion of the proof obligations.

Both conversion passes, steps (1) and (3), traverse the input term from the root and push conversion operators through arithmetic operators toward variables and constants. If we can show that no modular wraparound is possible for an operand, we do not have to account for overflow or underflow and thus end up with much simpler resulting terms.

For example, consider the `bvor` example from Sect. 2. For $B = 1$, when we are pushing `toNat` inward in `toNat( $x_0 + y_0 - x_0 \cdot y_0$ )`, if we can show that

$$\text{toNat}(x_0 \cdot y_0) \leq \text{toNat}(x_0 + y_0), \quad (6)$$

then we know the expression will not become negative (which would result in wraparound) and thus can soundly drop the modulus, executing the transformation:

$$\text{toNat}(x_0 + y_0 - x_0 \cdot y_0) \rightarrow \text{toNat}(x_0 + y_0) - \text{toNat}(x_0 \cdot y_0). \quad (7)$$

If we cannot prove Eq. (6), we have to account for possible wraparound, and our transformation becomes:

$$\text{toNat}(x_0 + y_0 - x_0 \cdot y_0) \rightarrow (\text{toNat}(x_0 + y_0) + P - \text{toNat}(x_0 \cdot y_0)) \bmod P, \quad (8)$$

which results in much larger bit widths that take longer to bit-blast. Therefore, we use different translation transformations (Sect. 4.2) for cases where modular wraparound must be accounted for and cases where it can be ruled out. To prove the absence of wraparound, we rely on range analysis.

A natural design choice would have been to rely on Lean’s existing automation, such as `ring_nf`, `omega`, or `aesop`, to discharge inequality goals. However, initial experiments showed that these tactics are often overwhelmed by large coefficients or struggle to handle variable dependencies and bound information arising in ZKP arithmetizations. We therefore design a dedicated range analysis algorithm.

Similar to the conversion passes, our range analysis, described in Sect. 4.3, is syntax-directed: it traverses the formula from the root toward variables and constants, introducing symbolic bounds for intermediate subterms. Leaf goals are discharged using bounds from the hypothesis context,⁴ and the resulting information is propagated back toward the root. We additionally incorporate specialized patterns that capture common bit-specific variable dependencies. While incomplete, our range analysis is largely independent of numeric magnitude and can

⁴ All variables have bounds as they are either finite field or bitvector variables.

$$\begin{array}{c}
\text{mvZModSub} \frac{t_1, t_2, t_3 : \mathbb{F}_P \quad \gamma \in \Gamma \quad t_1 - t_2 + t_3 \preceq \gamma}{\Gamma := \Gamma[t_1 - t_2 + t_3 / (t_1 + t_3) - t_2]} \\
\\
\text{injNat} \frac{t_1, t_2 : \mathbb{F}_P \quad t_1 = t_2 \in \Gamma}{\Gamma := \Gamma[t_1 = t_2 / \text{toNat}(t_1) = \text{toNat}(t_2)]} \\
\\
\text{distNat} \frac{t_1, t_2 : \mathbb{F}_P \quad \gamma \in \Gamma \quad \bowtie \in \{+, \cdot\} \quad \text{toNat}(t_1 \bowtie_{\mathbb{F}_P} t_2) \preceq \gamma}{\Gamma := \Gamma[\text{toNat}(t_1 \bowtie_{\mathbb{F}_P} t_2) / (\text{toNat}(t_1) \bowtie_{\mathbb{N}} \text{toNat}(t_2)) \bmod P]} \\
\\
\text{dist} \frac{t_1, t_2 : \mathbb{F}_P \quad t_3 : \text{Bool} \quad \gamma \in \Gamma \quad \text{toNat}(\text{ite}(t_3, t_1, t_2)) \preceq \gamma}{\text{NatIte} \quad \Gamma := \Gamma[\text{toNat}(\text{ite}(t_3, t_1, t_2)) / \text{ite}(t_3, \text{toNat}(t_1), \text{toNat}(t_2))]} \\
\\
\text{dist} \frac{t_1, t_2 : \mathbb{F}_P \quad \gamma \in \Gamma \quad \text{toNat}(t_1 - t_2) \preceq \gamma \quad H \models \text{toNat}(t_1) \geq \text{toNat}(t_2)}{\text{NatSub} \quad \Gamma := \Gamma[\text{toNat}(t_1 - t_2) / \text{toNat}(t_1) - \text{toNat}(t_2)]} \\
\\
\text{distNat} \frac{t_1, t_2 : \mathbb{F}_P \quad \gamma \in \Gamma \quad \text{toNat}(t_1 - t_2) \preceq \gamma}{\text{SubOvrflw} \quad \Gamma := \Gamma[\text{toNat}(t_1 - t_2) / (\text{toNat}(t_1) + P - \text{toNat}(t_2)) \bmod P]} \\
\\
\text{addBds} \frac{v_1 : \mathbb{F}_P \quad v_1 \preceq \gamma \quad \gamma \in \Gamma \quad \text{toNat}(v_1) \leq N \notin H \text{ for } N \in [0, P-1]}{H := H \cup \{\text{toNat}(v_1) \leq P-1\}} \\
\\
\text{mvMod} \frac{t_1, t_2, t_3 : \mathbb{N} \quad \gamma \in \Gamma \quad \bowtie \in \{+, \cdot\} \quad (t_1 \bmod t_3 \bowtie t_2 \bmod t_3) \bmod t_3 \preceq \gamma}{\Gamma := \Gamma[(t_1 \bmod t_3 \bowtie t_2 \bmod t_3) \bmod t_3 / (t_1 \bowtie t_2) \bmod t_3]} \\
\\
\text{dropMod} \frac{P \in \mathbb{N}_{>0} \quad t_1 : \mathbb{N} \quad t_1 \bmod P \preceq \gamma \quad \gamma \in \Gamma \quad H \models t_1 \leq P-1}{\Gamma := \Gamma[t_1 \bmod P / t_1]} \\
\\
\text{sqrBds} \frac{t_1 : \mathbb{F}_P \quad t_1 \cdot t_1 = t_1 \in \Gamma}{\Gamma := \Gamma[t_1 \cdot t_1 = t_1 / \text{toNat}(t_1) \leq 1]}
\end{array}$$

Fig. 2. Translation rules from $\mathbb{F}_P$ to $\mathbb{N}$. $\Gamma = \{G, H\}$ denotes the proof context, where G is the set of goals and H is the global set of shared hypotheses.

be more efficient than bit-blasting for large bit widths. Thus, we also use range analysis to discharge $\Sigma_{\mathbb{N}}$ -inequalities present in the original goals in step (2).

The theorem is proven once range analysis discharges all inequalities and bit-blasting discharges the final bitvector equality. Otherwise, we return **unknown**.

4.2 Translating from Finite Fields to Bitvectors

In this section, we present our *translation calculus*, which contains two types of rules: rules that convert from finite fields to natural numbers (Fig. 2) and rules that convert from natural numbers to bitvectors (Fig. 3).

We follow the common practice of presenting rules that modify *configurations* [32, 37]. In our calculus, a configuration is a proof context Γ , represented as a tuple (G, H) , where G is a set of goals and H is a set of global hypotheses shared across all goals. When a rule modifies only G or only H we omit the

injBV	$\frac{N \in \mathbb{N}_{>0} \quad t_1, t_2 : \mathbb{N} \quad t_1 = t_2 \in \Gamma \quad H \models t_1 \leq 2^N - 1 \quad H \models t_2 \leq 2^N - 1}{\Gamma := \Gamma[t_1 = t_2 / \text{toBV}(N, t_1) = \text{toBV}(N, t_2)]}$
injBV LeqHyp	$\frac{N \in \mathbb{N}_{>0} \quad t_1, t_2 : \mathbb{N} \quad \bowtie \in \{\leq, \geq\} \quad t_1 \bowtie_{\mathbb{N}} t_2 \in H \quad \text{toBV}(N, t_1) \bowtie_{[N]} \text{toBV}(N, t_2) \notin H \quad H \models t_1 \leq 2^N - 1 \quad H \models t_2 \leq 2^N - 1}{H := H \cup \{\text{toBV}(N, t_1) \bowtie_{[N]} \text{toBV}(N, t_2)\}}$
distBV	$\frac{N \in \mathbb{N}_{>0} \quad t_1, t_2 : \mathbb{N} \quad \gamma \in \Gamma \quad \bowtie \in \{+, \cdot\} \quad \text{toBV}(N, t_1 \bowtie_{\mathbb{N}} t_2) \preceq \gamma}{\Gamma := \Gamma[\text{toBV}(N, t_1 \bowtie_{\mathbb{N}} t_2) / \text{toBV}(N, t_1) \bowtie_{[N]} \text{toBV}(N, t_2)]}$
distBV Ite	$\frac{N \in \mathbb{N}_{>0} \quad t_1, t_2 : \mathbb{N} \quad t_3 : \text{Bool} \quad \gamma \in \Gamma \quad \text{toBV}(N, \text{ite}(t_3, t_1, t_2)) \preceq \gamma}{\Gamma := \Gamma[\text{toBV}(N, \text{ite}(t_3, t_1, t_2)) / \text{ite}(t_3, \text{toBV}(N, t_1), \text{toBV}(N, t_2))]}$
distBVSub	$\frac{N \in \mathbb{N}_{>0} \quad t_1, t_2 : \mathbb{N} \quad \gamma \in \Gamma \quad \text{toBV}(N, t_1 - t_2) \preceq \gamma \quad H \models t_1 \leq 2^N - 1 \quad H \models t_2 \leq t_1}{\Gamma := \Gamma[\text{toBV}(N, t_1 - t_2) / \text{toBV}(N, t_1) - \text{toBV}(N, t_2)]}$
distBVMod	$\frac{C, N \in \mathbb{N}_{>0} \quad t_1 : \mathbb{N} \quad \gamma \in \Gamma \quad C \leq 2^N - 1 \quad H \models t_1 \leq 2^N - 1}{\Gamma := \Gamma[\text{toBV}(N, t_1 \bmod C) / \text{toBV}(N, t_1) \bmod \text{toBV}(N, C)]}$

Fig. 3. Translation rules from $\mathbb{N}$ to BV_N .

unchanged component. Rules are presented in *guarded assignment form*, where the premises describe the configurations to which the rule can be applied, and the conclusion describes the modification to the configuration. A sequence of applications is called a *derivation*. We now present the two types of rules.

Finite Fields to Natural Numbers. **mvZModSub** groups additions before subtractions to accumulate positive values and ensure minimal underflow cases. **injNat** replaces finite field equations with natural number equations. **distNat** distributes the **toNat** operator over addition and multiplication and adds **mod** for soundness. Similarly **distNatIte** distributes the operator over if/then/else.

distNatSub distributes the **toNat** operator over subtraction when no negative values are possible. The **mod** operator can be soundly dropped as **toNat**(t_1) ensures the starting expression is less than P and subtraction by another natural number will only lower the final value. In cases where the premises of **distNatSub** cannot be discharged, we rely on **distNatSubOvrflw**, which ensures soundness by eliminating the possibility of negative values by adding P and then ensuring the result is in bounds by applying **mod** P . **addBds** adds bounds for finite field variables present in the proof context to ensure equisatisfiability of the natural number constraints. **mvMod** simplifies the term by factoring out **mod**. When the term is smaller than the modulus, we apply **dropMod** to remove the modulus.

While the majority of ZKP arithmetizations only use finite field variables to represent bits, the restriction that a variable must be 0 or 1 relies on finite field reasoning (finite fields' not having zero divisors), because finite fields do not

In: Proof context $\Gamma = (G, H)$	In: Proof context $\Gamma = (G, H)$
Out: Modified Γ	Out: Modified Γ
<pre> 1 def to_ℕ(Γ): 2 progress ← True 3 while progress do 4 progress ← (sqrBds injNat mvZModSub distNat distNatIte distNatSub distNatSubOvrflw mvMod dropMod addBds) </pre>	<pre> 1 def to_BV(Γ): 2 b ← 2 3 for γ ∈ Γ do 4 for t ≤ γ do 5 b ← max(c1cBVwidth(t), b) 6 progress ← True 7 while progress do 8 progress ← injBVLeqHyp(b) 9 progress ← True 10 while progress do 11 progress ← (injBV(b) distBV distBVite distBVSub distBVMod) </pre>
(a) Strategy to_ℕ	(b) Strategy to_BV

Fig. 4. Strategies for applying translation rules from Figs. 2 and 3.

naturally support comparisons or the Boolean operator **or**. Thus, we add **sqrBds** to detect bit encodings and translate them to natural number inequalities.

Natural Numbers to Bitvectors. As long as both sides of the expression fit into a bitvector of width N without overflow, **injBV** replaces natural number equalities with bitvector equalities of bit width N . While there might be inequalities, in our goal we leave them to range analysis and do not translate them. For hypothesis inequalities, we use **injBVLeqHyp** to maintain both the natural number and bitvector representations, so the inequalities can be used by the range analysis algorithm and by bit-blasting. **distBV** distributes the **toBV** operator over addition and multiplication. Unlike **distNat** there is no need to add a mod operator, as we remain in the mod 2^N space. **distBVite** distributes the **toBV** operator over if/then/else. **distBVSub** distributes the **toBV** operator over subtraction when no negative values are possible. Unlike **distNatSub**, we add a second check that $t_1 \leq 2^N - 1$ because t_1 is not naturally a bitvector and might overflow, while in **distNatSub**, t_1 is a **ZMod P** and is thus constrained to be less than or equal to $P - 1$. We do not include a rule for cases where negative values are possible, since this case will never arise from the finite field to natural number translation rules. Last but not least, both **dropMod** and **distBVMod** handle cases where the natural number encoding contains the mod operator. In cases where we cannot prove that the term is smaller than the modulus (**dropMod** does not apply) we apply **distBVMod** and distribute **toBV** as long as both the modulus and the term are smaller than 2 to the power of the bit width.

Strategy. Figure 4 presents our strategy, shown as pseudocode in which we assume that each translation rule returns **True** iff it applies and changes Γ . We also assume that **injBV** and **injBVLeqHyp** take as input the target bit width.

$(s_1 \parallel \dots \parallel s_k)$ is evaluated from left to right, stopping after the first **True** result. We first execute calculus rules according to Strategy **to_N** and then Strategy **to_{BV}**. **clcBVWidth**(t) is a function that overapproximates the maximum possible bit width needed to ensure t can be represented without overflow.⁵

- **to_N** The strategy consists of a while loop, where a rule is only attempted once the previous rule no longer applies. We begin with **sqrBds** to translate any bit encodings to bounds in case other rules rely on them. We then apply **injNat** and use **mvZModSub** to minimize underflow cases. Then, while one of **distNat**, **distNatIte**, **distNatSub**, or **distNatSubOvrflw** applies, we continue pushing **toNat** inward. We always attempt **distNatSub** before **distNatSubOvrflw** to ensure the final translation is as simple as possible. Once we reach saturation, we factor out the mod operations that came from distribution. Once no more mod operations can be factored we attempt to drop all mod operators (line 4).

- **to_{BV}** We begin by setting the maximum bit width b for the conversion to 2 (line 2). Then, for each hypothesis and goal we update b if any subterm requires a larger bit width (line 5). We calculate the maximum bit width across all of the terms to have a higher chance of proving the goal. For example, let $x \in \mathbb{N}$, and suppose we translate a hypothesis $x \leq 1$ to **toBV**(2, x) $\leq 1_{[2]}$ but then use **toBV**(4, x) in the goal. The goal then no longer follows from the hypothesis. A possible counterexample could include $x = 3_{[4]}$ as $(3_{[4]})_{[2]} = 1_{[2]}$. However, the value 3 would not have been permitted by the original hypothesis.

Upon establishing a maximum bit width, we use it to convert all natural numbers to bitvectors. We first apply **injBVLeqHyp** to saturation (to avoid applying it again if the converted inequalities are later modified, line 8), then cast goals to bitvectors using **injBV**, and then push **toBV** inward until saturation (line 11).

We state soundness and termination results below.

Theorem 1. *Termination: For any context Γ , no infinite derivation is possible using strategies **to_N** and **to_{BV}**.*

Theorem 2. *Soundness: For any context Γ , the bitvector translation Γ' produced by **to_N** and **to_{BV}** from Γ is valid if and only if Γ is valid.*

4.3 Range Analysis

We now describe the range analysis algorithm that handles $\Sigma_{\mathbb{N}}$ inequalities. The algorithm is presented as a calculus in Figs. 5-7. At a high level, the algorithm introduces new placeholder variables and applies lemmas to rewrite goals to formulas that can be inferred from the hypotheses. It also detects cases where variable dependencies influence these bounds and enumeration is feasible, and discharges the resulting goals by enumeration. We first show how the algorithm works on a simple modulus removal goal that arises during translation of the example from Sect. 2 and then explain the rules in more detail.

⁵ In our implementation, the bit width is calculated per variable, but we use a global bit width here to keep the presentation simple.

introPVar	$\frac{t_1, t_2 : \mathbb{N} \quad \bowtie \in \{\leq, \geq\} \quad \gamma \in G \quad \text{origOnly}(t_1 \bowtie t_2) \quad t_1 \bowtie t_2 \equiv \gamma}{G := (G \setminus \gamma) \cup \{t_1 \bowtie w, t_2 (\{\leq, \geq\} \setminus \bowtie) w\}}$
leqAddMul	$\frac{t_1, t_2 : \mathbb{N} \quad \bowtie \in \{+, \cdot\} \quad \gamma \in G \quad \text{origOnly}(t_1 \bowtie t_2) \quad t_1 \bowtie t_2 \leq w \equiv \gamma}{G := (G \setminus \gamma) \cup \{t_1 \leq w_1, t_2 \leq w_2, w_1 \bowtie w_2 \leq w\}}$
leqSub	$\frac{t_1, t_2 : \mathbb{N} \quad \gamma \in G \quad \text{origOnly}(t_1 - t_2) \quad t_1 - t_2 \leq w \equiv \gamma}{G := (G \setminus \gamma) \cup \{t_1 \leq w\}}$
leqMod	$\frac{t_1, t_2 : \mathbb{N} \quad \gamma \in G \quad \text{origOnly}(t_1 \bmod t_2) \quad t_1 \bmod t_2 \leq w \equiv \gamma}{G := (G \setminus \gamma) \cup \{t_2 \leq w\}}$
leq If	$\frac{t_3 : \mathbb{B} \quad t_1, t_2 : \mathbb{N} \quad \gamma \in G \quad \text{origOnly}(t_1) \quad \text{origOnly}(t_2) \quad \text{ite}(t_3, t_1, t_2) \leq w \equiv \gamma}{G := (G \setminus \gamma) \cup \{t_1 \leq w_1, t_2 \leq w_2, \max(w_1, w_2) \leq w\}}$

Fig. 5. Range analysis rules for inequality decomposition. We use w and w_i for placeholder variables and assume that any new placeholder variables in the conclusion are fresh. Other notation follows Fig. 2.

Motivating Example Revisited. Recall that our hypothesis state includes the formulas $\{\text{toNat}(x_0) \leq 1, \text{toNat}(y_0) \leq 1\}$. One goal that arises when processing the conclusion is: $\text{toNat}(x_0) + \text{toNat}(y_0) \leq P - 1$. To prove such goals, we introduce placeholder variables, which are implicitly existentially quantified. We then obtain formulas that can be unified with hypotheses by instantiating the placeholder variables. The evolution of the goal state is shown below.

$$\begin{aligned}
G_1 &= \{ \text{toNat}(x_0) + \text{toNat}(y_0) \leq w_1, P - 1 \geq w_1 \} \\
G_2 &= \{ \text{toNat}(x_0) \leq w_2, \text{toNat}(y_0) \leq w_3, w_2 + w_3 \leq w_1, P - 1 \geq w_1 \} \\
G_3 &= \{ 1 + 1 \leq w_1, P - 1 \geq w_1 \} \\
G_4 &= \{ 1 + 1 \leq P - 1 \}
\end{aligned}$$

The transition from G_2 to G_3 uses the hypotheses in H to match the pattern in the goal, resulting in the concrete bounds obtained in G_3 .

To automate the inequality reasoning we formulate a calculus made up of three types of rules: Rules that decompose inequalities using placeholder variables (Fig. 5), rules that remove placeholder variables (Fig. 6), and rules that reason about bits and constants (Fig. 7).

To help describe the rules, we introduce additional notation and helper functions. We use *original variables*, denoted by the set **origVars**, to describe variables that exist in the proof context *before* starting range analysis, and *placeholder variables*, denoted by the set **pVars**, to describe *implicitly existentially quantified* variables introduced by the range analysis. **origOnly**(t) is true iff all variables occurring in t (of which there must also be at least one) are not in **pVars**. **twoOrigVars**(t) is true iff a term contains exactly 2 distinct variables not in **pVars**. **hasSub**(t) is true iff t contains at least one subtract ($-_{\mathbb{N}}$) operator.

$$\begin{array}{c}
\text{C} \in \mathbb{N} \quad t_1 : \mathbb{N} \quad \bowtie \in \{\leq, \geq\} \\
\text{ineqHyp} \frac{\gamma \in G \quad \text{origOnly}(t_1) \quad t_1 \bowtie w \equiv \gamma \quad t_1 \bowtie C \in H}{G := (G \setminus \{\gamma' \mid \gamma' \in G \wedge w \preceq \gamma'\}) \cup \{\gamma' \mid w/C \mid \gamma' \in G \setminus \gamma\}} \\
\\
\text{leqZMod} \frac{v : \mathbb{F}_P \quad v \notin \text{pVars} \quad \text{toNat}(v) \leq w \equiv \gamma \quad \gamma \in G}{G := (G \setminus \{\gamma' \mid \gamma' \in G \wedge w \preceq \gamma'\}) \cup \{\gamma' \mid w/P - 1 \mid \gamma' \in G \setminus \gamma\}} \\
\\
\text{leqBV} \frac{N \in \mathbb{N}_{>0} \quad v : \text{BV}_{[N]} \quad v \notin \text{pVars} \quad \text{bvToNat}(v) \leq w \equiv \gamma \quad \gamma \in G}{G := (G \setminus \{\gamma' \mid \gamma' \in G \wedge w \preceq \gamma'\}) \cup \{\gamma' \mid w/2^N - 1 \mid \gamma' \in G \setminus \gamma\}} \\
\\
\text{geNat} \frac{t_1 : \mathbb{N} \quad \text{origOnly}(t_1) \quad t_1 \geq w \equiv \gamma \quad \gamma \in G}{G := (G \setminus \{\gamma' \mid \gamma' \in G \wedge w \preceq \gamma'\}) \cup \{\gamma' \mid w/0 \mid \gamma' \in G \setminus \gamma\}} \\
\\
\text{ineqConst} \frac{\text{C} \in \mathbb{N} \quad \bowtie \in \{\leq, \geq\} \quad \gamma \in G \quad C \bowtie w \equiv \gamma \quad \forall \gamma' \in G. (w \not\preceq \gamma' \vee \forall v \preceq \gamma'. v \notin \text{origVars})}{G := (G \setminus \{\gamma' \mid \gamma' \in G \wedge w \preceq \gamma'\}) \cup \{\gamma' \mid w/C \mid \gamma' \in G \setminus \gamma\}}
\end{array}$$

Fig. 6. Range analysis rules for placeholder variable elimination. Notation follows that of Fig. 5.

$$\begin{array}{c}
\text{eval} \frac{C_1, C_2 \in \mathbb{N} \quad \gamma \in G \quad C_1 \leq C_2 \equiv \gamma}{G := (G \setminus \gamma) \cup \{\text{eval}(\gamma)\}} \\
\\
t_1, t_2, t_3, v_0, v_1 : \mathbb{N} \quad \bowtie \in \{\leq, \geq\} \\
\gamma \in G \quad t_1 \bowtie t_2 \equiv \gamma \quad v_0, v_1 \preceq \gamma \quad \text{twoOrigVars}(\gamma) \\
\text{ineq} \frac{\text{hasSub}(\gamma) \vee \text{origOnly}(t_1) \wedge \text{origOnly}(t_2) \quad v_0 \leq 1 \in H \quad v_1 \leq 1 \in H}{\text{Cases} \quad G := (G \setminus \gamma) \cup \{\gamma[v_1, v_0/0, 0], \gamma[v_1, v_0/1, 0], \gamma[v_1, v_0/0, 1], \gamma[v_1, v_0/1, 1]\}} \\
\\
N \in \mathbb{N}_{>0} \quad v_1, t_1, t_2 : \mathbb{N} \quad \bowtie \in \{\leq, \geq\} \\
\text{ineqXOR} \frac{\gamma \in G \quad v_1 \cdot_{\mathbb{N}} t_1 + (1 - v_1) \cdot_{\mathbb{N}} t_2 \bowtie w \equiv \gamma \quad v_1 \leq 1 \in H}{G := (G \setminus \gamma) \cup \{\text{ite}(v_1 = 0, t_2, t_1) \bowtie w\}}
\end{array}$$

Fig. 7. Range analysis rules for constant and bit reasoning. Notation follows that of Fig. 5.

Inequality Decomposition. **introPVar** introduces a placeholder variable into any inequality that contains at least one original variable and no placeholder variables. We ensure that placeholder variables are on the right-hand side of inequalities when original variables or constants are present in the expression, to make decomposition and elimination easier. **leqAddMul** and **leqIf** rely on standard interval arithmetic [16] to introduce additional placeholder variables into the context. **leqSub** strengthens the goal by ignoring t_2 , which is reasonable since in practice t_2 rarely has a lower bound other than zero. Finally, **leqMod** similarly strengthens the goal by keeping only the modulus.

Placeholder Variable Elimination. After decomposition, we eliminate placeholder variables by instantiating them with bounds derived from the context. The rule `ineqHyp` instantiates placeholder variables using inequalities present in the hypotheses. The rule `leqZMod` uses the fact that finite field values are bounded from above by the field modulus, `leqBV` uses the fact that N -bit bitvectors are bounded above by $2^N - 1$, and `geNat` relies on the fact that natural numbers are always greater than or equal to zero. If a placeholder variable occurs in an inequality with a constant, we instantiate the placeholder variable with the constant itself using `ineqConst`, but only if all other occurrences of the placeholder variable do not include original variables. The check ensures we do not get into a rule application cycle.

Constant and Bit Reasoning. Once all placeholder and original variables are eliminated from a goal, we can use `eval` to check if the derived constant term is true. Since the inequality decomposition rules are intentionally syntax-directed and do not account for dependencies between variables, we add rules that detect and resolve common dependency patterns in ZKP arithmetizations. `ineqCases` identifies potential dependence between two bit-simulating variables v_0 and v_1 . We restrict this rule to the setting when only two variables are present to keep the case analysis manageable. We conservatively flag a dependency when either (i) the goal contains a subtraction operator, or (ii) both sides of the inequality contain variables. When a potential dependency is detected, `ineqCases` performs explicit case analysis over the four possible assignments to v_0 and v_1 . `ineqXOR` handles encodings that simulate XORs; if a bit-simulating variable is used as guard, the goal can be turned into an if/then/else expression.

Due to its inability to detect and solve all variable dependencies and the fact that our range analysis sometimes strengthens the goal, it is not complete (e.g., `eval` might return `False` for a valid inequality) but it is terminating and sound. We state the theorems below and include the proofs in the extended version of the paper [33] due to space constraints.

Theorem 3. *Termination: For any context Γ , there is no infinite derivation from the range analysis calculus.*

Theorem 4. *Soundness: For any context Γ , if a derivation from Γ reaches a context where the set of goals is empty, then all goals in Γ hold.*

4.4 Implementation

We implement `BitModEq` in Lean v4.23.0. We use `bv_decide` [8] for bit-blasting and `mathlib` [21] for key lemmas. The majority of our translation and range analysis rules are implemented using existing lemmas in `mathlib`. We proved additional lemmas for the rules: `distNatIte`, `distNatSubOvrflw`, `addBds`, `sqrBds`, `injBVLeqHyp`, `distBVIte`, `distBVSub`, `distBVMod`, `leqSub`, and `ineqXOR` (Figs. 2, 3, 5 and 7). To reliably pattern match and apply lemmas, we require formulas to be in a normalized form. We rely on Lean’s `simp`, `split_ifs`,

Table 3. BitModEq and cvc5 comparison for Lasso and T&S benchmarks.

Solver	Lasso T&S		Solver	Lasso	T&S
BitModEq	24	12	BitModEq	0	0
cvc5 _{weak}	21	5	SMT _{weak}	0	0
cvc5-split _{weak}	10	2	SMT _{strong}	9	15
cvc5 _{strong}	4	0	# Bench.	24	15
cvc5-split _{strong}	3	0	(b) Statement generation timeouts		
# Bench.	24	15			

(a) Solved instances

and `split_and`s to simplify the syntax of formulas and add rewrites for commutativity, associativity and the negation operator. When doing range analysis, we always first try to apply `ineqCases`, `ineqXOR`, `ineqHyp` and `ineqConst` to have a better chance of proving the goal. `geNat` is only used when no other rules apply.

We rely on Lean’s metavariable mechanism (introduced as part of lemma hypotheses) to represent placeholder variables and on Lean’s kernel to eliminate them throughout the proof context when applying placeholder elimination rules (Fig. 6). The pipeline of BitModEq can involve proving similar inequalities (e.g., when $P > 2^N$, one way to prove $t_1 \bmod P \leq 2^N - 1$ is to prove $t_1 \leq P - 1 \wedge t_1 \leq 2^N - 1$). Thus, to prove the soundness of the bitvector conversion step, when $P > 2^N$ we prove $t_1 \leq 2^N - 1$ and use it to solve both goals.

5 Evaluation

Our experiments answer two key empirical questions: (1) How does BitModEq compare to the state of the art on ZKP arithmetization benchmarks? (2) How critical is the proposed range analysis algorithm to BitModEq’s effectiveness? All experiments are run on a cluster with Intel Xeon E5-2637 v4 CPUs.

5.1 Benchmarks

To evaluate the effectiveness of BitModEq, we verify ZKP arithmetizations extracted from two real-world systems described below. The evaluated systems are representative of the types of arithmetizations commonly used in many modern ZKP frameworks, including Cheesecloth and Ligetron [10, 11, 41].

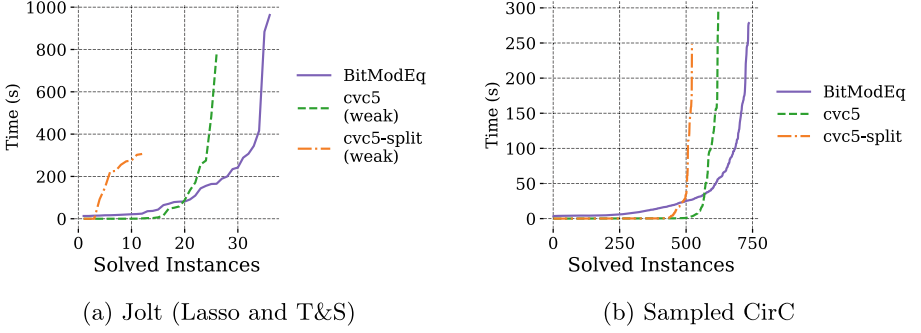


Fig. 8. Cactus plots across benchmark suites comparing BitModEq and cvc5

Table 4. BitModEq and cvc5 comparison on CirC benchmarks (0–6 arguments; 0–32 bits), sampling up to two benchmarks per even bit width and operation (i.e., $\min(2, n)$ where n is the number of benchmarks available). Extraction checks are not generated at 32 bits, as this is the base width for the operator.

Solver	Bit width																	All
	0	2	4	6	8	10	12	14	16	18	20	22	24	26	28	30	32	
BitModEq	34	50	51	50	49	50	47	43	41	42	40	42	44	38	41	40	35	737
cvc5	40	78	71	54	38	35	33	34	30	27	26	25	27	26	27	27	23	621
cvc5-split	40	74	61	47	32	31	28	27	22	21	20	20	21	20	20	21	17	522
# Bench.	40	80	80	80	80	80	80	80	80	80	80	80	80	80	80	80	78	1318

Jolt. Jolt [2] is a zkVM that relies on lookup tables to encode RISC-V instructions as finite field polynomials. The original Jolt design uses Lasso [36] to perform 8-bit input lookup tables, while subsequent work, Twist&Shout [35], extends this approach to 32-bit input tables. We extract the lookup table arithmetizations from both systems and verify their semantic equivalence to corresponding Lean bitvector operations.⁶ We include a breakdown of benchmark size and complexity in the extended version of the paper [33].

CirC. CirC [27] is a zkVM compiler whose arithmetization passes were previously verified using cvc5 [30]. We implement a Lean backend for arithmetization verification in CirC and generate benchmarks at larger bit-widths than those considered in prior work.

5.2 State of the Art Comparison

Baselines. Since there is no prior automation in Lean that works on ZKP arithmetizations, we compare BitModEq against cvc5’s [4] two main finite field decision procedures, one based on full Gröbner bases [28] and the other on split

⁶ We omit three Twist&Shout instructions which depend on external assumptions.

Table 5. Ablation study of the **BitModEq** range analysis procedure and comparison against existing Lean automation tactics.

Solver	Lasso	T&S	CirC
BitModEq w/ full range analysis	24	12	737
BitModEq w/o case splits	21	8	720
BitModEq w/ lean tactics	7	4	434
# Bench.	24	15	1318

Gröbner bases [29] (v.1.3.2), on benchmarks encoded in the SMT finite field and bitvector theories. For Jolt we evaluate the *weak* and *strong* SMT encodings, as discussed in Sect. 2. For CirC benchmarks, we rely on the existing SMT backend, where the overflow checks are checked by the compiler [30]. In contrast, our Lean encoding always maintains the stronger guarantees. If generation of a benchmark for any encoding takes more than a minute we consider it a timeout.

Jolt. We run the Jolt benchmarks with a 20 min timeout and 16GB memory limit. As summarized in Table 3a, **BitModEq** solves three more benchmarks than the weak **cvc5** encoding for the 8-bit lookup tables, and seven more for the 32-bit lookup tables. Table 3b shows that the strong SMT encoding times out during generation for 24 benchmarks, all of which involve range checks with large constants (2^{32}). The weak encoding does not timeout as it avoids the range check.

The cactus plot in Fig. 8a shows that while **cvc5** solves simpler benchmarks faster (e.g., **msb**, **lsb** arithmetization), **BitModEq** significantly outperforms **cvc5** on larger benchmarks while also producing kernel-checked proofs, and despite operating under a stronger encoding.

CirC. Using the pipeline from prior work [30], we generate CirC benchmarks covering arithmetization passes with 0 to 6 arguments using bit widths from 0 to 32. Due to the large number of benchmarks, we focus on even bit widths, sample up to two benchmarks per arithmetization per bit width, and run with a five minute timeout and 8GB memory limit. As shown in Table 4, **cvc5** outperforms **BitModEq** on lower bit widths but scales poorly to the larger ones. **BitModEq** solves more benchmarks for 13 out of the 17 bit widths.

In general, **BitModEq** performs well on constraints that simulate bit-level semantics with tightly bounded variables but is less effective on constraints with unbounded variables or heavy use of field-specific operations (e.g., inverses, negation). Using large field constants, multiplicative inverse constraints (e.g., equations of the form $x \cdot y = 1$), and field negation as an overapproximation, we estimate that 42% of the CirC benchmarks contain some field-specific behavior.⁷ These behaviors include arithmetizations corresponding to bitvector operators such as **bvnot** and **bvs1e**. Although **cvc5**'s theory combination approach is more general and performs better when field-specific operations are present,

⁷ In contrast, none of the Jolt benchmarks rely on field-specific behavior.

its bit-splitting strategy introduces a large number of auxiliary variables, which significantly degrades performance at higher bit widths.

Across the benchmark suite, 130 SMT instances timeout during encoding generation. We include the breakdown in the extended version of the paper [33]. Similarly to the Jolt benchmarks, as seen in Fig. 8b, BitModEq is slower on simpler benchmarks but outperforms cvc5 on larger ones.

5.3 Range Analysis Evaluation

To evaluate the effectiveness of our range analysis we substitute the algorithm for a portfolio of existing Lean automation tactics: `ring_nf`, `linarith`, `nlinarith`, `aesop`, `grind`, and `omega`. We additionally evaluate the impact of disabling case splitting (`ineqCases` and `ineqXOR`). Table 5 shows the results. BitModEq with the full range analysis solves substantially more benchmarks across all benchmark sets. Case splits play a more significant role for the Jolt benchmarks because they tend to contain more variable dependencies.

5.4 Discovered Bugs

During CirC evaluation, we identified a soundness bug in the 32-bit arithmetic right shifts (`bvashr`) related to integer overflow in CirC’s Rust implementation. The issue was uncovered via SMT-based verification during tool comparison, did not occur at smaller bit widths, and shows the importance of verification beyond toy sizes. The CirC developers have been notified and have fixed the issue.

6 Related Work

Prior work on verifying ZKP arithmetizations broadly falls into two categories: manual verification in ITPs and automatic verification using SMT solvers. Manual approaches provide strong end-to-end correctness guarantees but require substantial proof engineering effort. For example, [39] proves the correctness of the SP1 Hypercube zkVM instruction chips with respect to Sail’s RISC-V semantics [1], and [3] verifies that Cairo’s AIR correctly constrains execution traces. Automatic approaches aim to reduce manual effort by relying on SMT solvers. Özdemir et al. [30] verify CirC’s finite-field-blasting pipeline using cvc5, checking all compiler arithmetizations from 0 to 4 arguments and 0 to 4 bits. While automated, this approach maintains large encodings and struggles to scale.

Beyond arithmetization pipelines, a large body of work focuses on the verification of ZK circuits themselves. On the proof-assistant side, Liu et al. [20] present CODA, a circuit language based on refinement types for specifying and proving circuit properties, while the CIVER tool [18] defines transformation and deduction rules for verifying Circom circuits. On the SMT side, *QED*² [31] detects underconstrained circuits using finite field reasoning, and Zequal [38] checks consistency between circuits and witness generation. Complementary fuzzing-based

approaches have also been effective at uncovering bugs in ZK circuits and compilers [9, 40, 42]. While valuable for improving security, these techniques focus on circuit-level properties and do not establish end-to-end correctness guarantees. As demonstrated by bounded verification of CirC’s finite field blasting passes [30], correctness issues can occur at the level of arithmetization rules themselves, independent of the generated circuit. Another closely related work is the verification of Lasso lookup tables by Kwan et al. [19] in ACL2, which also relies on bit-blasting. Unlike our work, this work relies on ACL2’s existing correspondence between finite fields and bitvectors and represents lookup tables as bits instead of finite field elements.

Finally, our approach connects to prior work on SMT theory translations. Zohar et al. [43] translate bitvector constraints to integers, while Pertseva et al. [32] lift and lower constraints between integers and modular arithmetic using range analysis. However, these works do not produce kernel-checked proofs in an ITP and do not simultaneously reason across finite field and bitvector theories, which is essential for verifying ZKP arithmetizations.

7 Conclusion and Future Work

This paper presents **BitModEq**, a Lean tactic for automatically proving equivalence between finite field and bitvector encodings present in ZKP arithmetizations. By combining translation, range analysis, and bit-blasting in Lean, **BitModEq** produces kernel-checked proofs and outperforms state-of-the-art solvers on real-world benchmarks. Multiple directions remain for future work. An important challenge is verifying systems along rarely exercised code paths, specifically addressing cases where bit widths exceed the modulus, which will require more scalable verification. It would also be valuable to characterize when translation to bitvectors is most effective and when direct theory combination or finite-field reasoning is preferable, as shown by CirC benchmarks. Finally, developing stronger native automation for finite field reasoning in Lean, including proof support for existing SMT-based decision procedures, remains crucial, as bitvector translation alone cannot capture all aspects of finite field semantics.

Acknowledgments. This work was supported in part by the Stanford Center for Automated Reasoning (Centauro) and a grant from the Ethereum Foundation.

Data Availability Statement. The Lean and SMT versions of the benchmarks, the implementation of **BitModEq**, and instructions for reproducing the results are available on [Zenodo](https://zenodo.org/doi/10.5281/zenodo.19841101) (DOI: 10.5281/zenodo.19841101).

Disclosure of Interest. Valentin Robert is employed at Galois. Elizaveta Pertseva and James Parker previously worked at Galois. Clark Barrett is an Amazon Scholar.

References

1. Armstrong, A., et al.: Isa semantics for armv8-a, risc-v, and cheri-mips. *Pro. ACM Programming Lang.* **3**(POPL), 1–31 (2019). <https://doi.org/10.1145/3290384>
2. Arun, A., Setty, S., Thaler, J.: Jolt: snarks for virtual machines via lookups. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 3–33. Springer (2024). https://doi.org/10.1007/978-3-031-58751-1_1
3. Avigad, J., Goldberg, L., Levit, D., Seginer, Y., Titelman, A.: A verified algebraic representation of cairo program execution. In: *Proceedings of the 11th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pp. 153–165 (2022). <https://doi.org/10.1145/3497775.3503675>
4. Barbosa, H., et al.: cvc5: a versatile and industrial-strength SMT solver. In: *TACAS* (2022). https://doi.org/10.1007/978-3-030-99524-9_24
5. Barbosa, H., et al.: Generating and exploiting automated reasoning proof certificates. *Commun. Assoc. Comput. Mach. (CACM)* **66**(10), 86–95 (2023). <https://doi.org/10.1145/3587692>
6. Barrett, C., Tinelli, C.: Satisfiability modulo theories. In: *Handbook of model checking*, pp. 305–343. Springer (2018). <https://doi.org/10.3233/FAIA201017>
7. Baum, C., et al.: Faest: algorithm specifications. Technical report, National Institute of Standards and Technology (2023)
8. Böving, H., et al.: Interactive bitvector reasoning using verified bit-blasting. *Proc. ACM Program. Lang.* **9**(OOPSLA2), 3259–3285 (2025). <https://doi.org/10.1145/3763167>
9. Chaliasos, S., Al-Fath, I., Donaldson, A.: Towards fuzzing zero-knowledge proof circuits (short paper). In: *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 98–104 (2025). <https://doi.org/10.1145/3713081.3731718>
10. Cuéllar, S., Harris, B., Parker, J., Pernsteiner, S., Tromer, E.: Cheesecloth: {Zero-Knowledge} proofs of real world vulnerabilities. In: *32nd USENIX Security Symposium (USENIX Security 23)*, pp. 6525–6540 (2023)
11. Gempeler, S.C., Harris, B., Parker, J., Pernsteiner, S., Sweet, I., Tromer, E.: Cheesecloth: zero-knowledge proofs of real-world vulnerabilities. *ACM Trans. Priv. Security* **28**(4), 1–35 (2025). <https://doi.org/10.1145/3747589>
12. Datta, T., Chen, B., Boneh, D.: Veritas: verifying image transformations at scale. In: *2025 IEEE Symposium on Security and Privacy (SP)*, pp. 4606–4623. IEEE (2025). <https://doi.org/10.1109/SP61157.2025.00097>
13. Dummit, D.S., Foote, R.M.: *Abstract algebra*, vol. 3. Wiley Hoboken (2004)
14. Enderton, H.B.: *A mathematical introduction to logic*. Elsevier (2001)
15. Hader, T., Ozdemir, A.: An SMT-LIB theory of finite fields. In: Reger, G., Zohar, Y (eds.) *Proceedings of the 22nd International Workshop on Satisfiability Modulo Theories co-located with the 36th International Conference on Computer Aided Verification (CAV 2024)*, Montreal, Canada, 22–23 July 2024, *CEUR Workshop Proceedings*, pages 3–12. CEUR-WS.org (2024). <https://ceur-ws.org/Vol-3725/paper3.pdf>
16. Hickey, T., Ju, Q., Van Emden, M.H.: Interval arithmetic: from principles to implementation. *J. ACM (JACM)* **48**(5), 1038–1068 (2001). <https://doi.org/10.1145/502102.502106>
17. Hopwood, D., Bowe, S., Hornby, T., Wilcox, N., et al.: Zcash protocol specification. GitHub: San Francisco, CA, USA, vol. 4(220), p. 32 (2016)

18. Isabel, M., Rodriguez-Nunez, C., Rubio, A.: Scalable verification of zero-knowledge protocols. In: 2024 IEEE Symposium on Security and Privacy (SP), pp. 1794–1812. IEEE (2024). <https://doi.org/10.1109/SP54263.2024.00133>
19. Kwan, C., Dao, Q., Thaler, J.: Verifying jolt zkvm lookup semantics. In: International Conference on Financial Cryptography and Data Security, pp. 163–179. Springer (2025). https://doi.org/10.1007/978-3-032-07024-1_10
20. Liu, J., et al.: Certifying zero-knowledge circuits with refinement types. In: 2024 IEEE Symposium on Security and Privacy (SP), pp. 1741–1759. IEEE (2024). <https://doi.org/10.1109/SP54263.2024.00078>
21. The mathlib Community. The lean mathematical library. In: Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2020, pp. 367–381. Association for Computing Machinery, New York, USA (2020). <https://doi.org/10.1145/3372885.3373824>, ISBN 9781450370974
22. McEliece, R.J.: Finite fields for computer scientists and engineers, vol. 23. Springer Science & Business Media (2012)
23. Miao, Y.: Secure and privacy-preserving voting system using zero-knowledge proofs. *Appl. Comput. Eng.* **8**(1), 328–333 (2023)
24. Mohamed, A., et al.: Lean-smt: an smt tactic for discharging proof goals in lean. In: International Conference on Computer Aided Verification, pp. 197–212. Springer (2025). https://doi.org/10.1007/978-3-031-98682-6_11
25. Moura, L., Ullrich, S.: The lean 4 theorem prover and programming language. In: Platzer, A., Sutcliffe, G. (eds.) CADE 2021. LNCS (LNAI), vol. 12699, pp. 625–635. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-79876-5_37
26. Nelson, G., Oppen, D.C.: Simplification by cooperating decision procedures. *ACM Trans. Program. Lang. Syst. (TOPLAS)* **1**(2), 245–257 (1979). <https://doi.org/10.1145/357073.357079>
27. Ozdemir, A., Brown, F., Wahby, R.S.: CirC: compiler infrastructure for proof systems, software verification, and more. In: IEEE S&P (2022). <https://doi.org/10.1109/SP46214.2022.9833782>
28. Ozdemir, A., Kremer, G., Tinelli, C., Barrett, C.: Satisfiability modulo finite fields. In: CAV (2023). https://doi.org/10.1007/978-3-031-37703-7_8
29. Ozdemir, A., Pailoor, S., Bassa, A., Ferles, K., Barrett, C., Dillig, I.: Split gröbner bases for satisfiability modulo finite fields. In: CAV (2024). https://doi.org/10.1007/978-3-031-65627-9_1
30. Ozdemir, A., Wahby, R.S., Brown, F., Barrett, C.: Bounded verification for finite-field-blasting in a compiler for zero knowledge proofs. *Formal Methods Syst. Des.* **67**(2), 161–188 (2025). <https://doi.org/10.1007/S10703-025-00476-3>
31. Pailoor, S., et al.: Automated detection of under-constrained circuits in zero-knowledge proofs. *Proc. ACM Program. Lang.* **7**(PLDI), pp. 1510–1532 (2023). <https://doi.org/10.1145/3591282>
32. Pertseva, E., et al.: Integer reasoning modulo different constants in smt. In: International Conference on Computer Aided Verification, pp. 339–362. Springer (2025). https://doi.org/10.1007/978-3-031-98668-0_17
33. Pertseva, E., Robert, V., Barrett, C., Parker, J.: Automating bitvector and finite field equivalence proofs in lean (2026). <https://arxiv.org/abs/2605.15163>
34. Ranise, S., Tinelli, C., Barrett, C.: SMT fixed size bit-vectors theory (2017). <https://smtlib.cs.uiowa.edu/theories-FixedSizeBitVectors.shtml>
35. Setty, S., Thaler, J.: Twist and shout: Faster memory checking arguments via one-hot addressing and increments. *Cryptology ePrint Archive*, Paper 2025/105 (2025). <https://eprint.iacr.org/2025/105>

36. Setty, S., Thaler, J., Wahby, R.: Unlocking the lookup singularity with lasso. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques, pp. 180–209. Springer (2024). https://doi.org/10.1007/978-3-031-58751-1_7
37. Sheng, Y., et al.: Reasoning about vectors using an smt theory of sequences. In: International Joint Conference on Automated Reasoning, pp. 125–143. Springer International Publishing Cham (2022). https://doi.org/10.1007/978-3-031-10769-6_9
38. Stephens, J., Pailoor, S., Dillig, I.: Automated verification of consistency in zero-knowledge proof circuits. In: International Conference on Computer Aided Verification, pp. 315–338. Springer (2025). https://doi.org/10.1007/978-3-031-98668-0_16
39. Succinct.xyz Blog. Formal verification of sp1 hypercube (2025). <https://blog.succinct.xyz/nethermind-lean/>. Accessed 12 July 2025
40. Takahashi, H., Kim, J., Jana, S., Yang, J.: zkfuzz: foundation and framework for effective fuzzing of zero-knowledge circuits. arXiv preprint [arXiv:2504.11961](https://arxiv.org/abs/2504.11961) (2025)
41. Wang, R., Hazay, C., Venkatasubramanian, M.: Ligetron: lightweight scalable end-to-end zero-knowledge proofs post-quantum zk-snarks on a browser. In: 2024 IEEE Symposium on Security and Privacy (SP), pp. 1760–1776. IEEE (2024). <https://doi.org/10.1109/SP54263.2024.00086>
42. Xiao, D., Liu, Z., Peng, Y., Wang, S.: Mtzk: testing and exploring bugs in zero-knowledge (zk) compilers. In: NDSS (2025)
43. Zohar, Y., et al.: Bit-precise reasoning via int-blasting. In: International Conference on Verification, Model Checking, and Abstract Interpretation, pp. 496–518. Springer (2022). https://doi.org/10.1007/978-3-030-94583-1_24

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (<http://creativecommons.org/licenses/by/4.0/>), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

